405finalcode

Shiyu Jiang

2026-04-18

```
library(cmdstanr)
```

```
## Warning: package 'cmdstanr' was built under R version 4.5.3
```

```
## This is cmdstanr version 0.9.0
```

```
## - CmdStanR documentation and vignettes: mc-stan.org/cmdstanr
```

```
## - CmdStan path: /Users/eyl/.cmdstan/cmdstan-2.38.0
```

```
## - CmdStan version: 2.38.0
```

```
library(MASS)
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
library(bayesplot)
```

```
## Warning: package 'bayesplot' was built under R version 4.5.2
```

```
## This is bayesplot version 1.15.0
```

```
## - Online documentation and vignettes at mc-stan.org/bayesplot
```

```
## - bayesplot theme set to bayesplot::theme_default()
```

```
##    * Does _not_ affect other ggplot2 plots
```

```
##    * See ?bayesplot_theme_set for details on theme setting
```

```
library(posterior)
```

```
## This is posterior version 1.6.1
```

```
##
## Attaching package: 'posterior'
```

```
## The following object is masked from 'package:bayesplot':
##
##     rhat
```

```
## The following objects are masked from 'package:stats':
##
##     mad, sd, var
```

```
## The following objects are masked from 'package:base':
##
##     %in%, match
```

#read data

```r
data <- read.csv("data405.csv", stringsAsFactors = FALSE)

data <- subset(
  data,
  !is.na(playerName) &
    !is.na(age) &
    !is.na(AB) &
    !is.na(H) &
    is.finite(age) &
    is.finite(AB) &
    is.finite(H) &
    AB > 0 &
    H >= 0 &
    H <= AB
)

data$playerName <- factor(data$playerName)
data$player_id <- as.integer(data$playerName)

age_center <- mean(data$age)
age_scale  <- sd(data$age)

data$age_c  <- as.numeric(scale(data$age, center = TRUE, scale = TRUE))
data$age_c2 <- data$age_c^2

J <- length(unique(data$player_id))

str(data)
```

```
## 'data.frame':    2577 obs. of  14 variables:
##  $ playerName : Factor w/ 200 levels "A. J. Pierzynski",..: 22 22 22 22 22 22 22 22 22 22 ...
##  $ birthYear  : int  1974 1974 1974 1974 1974 1974 1974 1974 1974 1974 ...
##  $ debutYear  : int  1996 1996 1996 1996 1996 1996 1996 1996 1996 1996 ...
##  $ finalYear  : int  2014 2014 2014 2014 2014 2014 2014 2014 2014 2014 ...
##  $ yearID     : int  1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 ...
##  $ age        : int  23 24 25 26 27 28 29 30 31 32 ...
##  $ age2       : int  529 576 625 676 729 784 841 900 961 1024 ...
##  $ AB         : int  188 497 546 576 588 572 577 574 588 548 ...
##  $ H          : int  47 155 183 182 170 176 173 173 168 163 ...
##  $ batting_avg: num  0.25 0.312 0.335 0.316 0.289 ...
##  $ n_seasons  : int  17 17 17 17 17 17 17 17 17 17 ...
##  $ player_id  : int  22 22 22 22 22 22 22 22 22 22 ...
##  $ age_c      : num  -1.611 -1.384 -1.157 -0.929 -0.702 ...
##  $ age_c2     : num  2.596 1.915 1.338 0.864 0.493 ...
```

```r
head(data[, c("playerName", "player_id", "age", "age_c", "age_c2", "AB", "H")])
```

```
##    playerName player_id age      age_c    age_c2  AB   H
## 1 Bobby Abreu        22  23 -1.6110947 2.5956261 188  47
## 2 Bobby Abreu        22  24 -1.3838359 1.9150018 497 155
## 3 Bobby Abreu        22  25 -1.1565771 1.3376706 546 183
## 4 Bobby Abreu        22  26 -0.9293183 0.8636325 576 182
## 5 Bobby Abreu        22  27 -0.7020595 0.4928875 588 170
## 6 Bobby Abreu        22  28 -0.4748007 0.2254357 572 176
```

#stan prepare

```
stan_data_simple <- list(
  N = nrow(data),
  AB = data$AB,
  H = data$H,
  age_c = data$age_c,
  age_c2 = data$age_c2
)

stan_data_hier <- list(
  N = nrow(data),
  J = J,
  player = data$player_id,
  AB = data$AB,
  H = data$H,
  age_c = data$age_c,
  age_c2 = data$age_c2
)
```

```
simple_stan_code <- "
data {
  int<lower=1> N;
  array[N] int<lower=0> AB;
  array[N] int<lower=0> H;
  vector[N] age_c;
  vector[N] age_c2;
}

parameters {
  real beta0;
  real beta1;
  real beta2;
}

model {
  vector[N] logit_theta;

  beta0 ~ normal(0, 5);
  beta1 ~ normal(0, 5);
  beta2 ~ normal(0, 5);

  logit_theta = beta0 + beta1 * age_c + beta2 * age_c2;

  H ~ binomial_logit(AB, logit_theta);
}

generated quantities {
  real peak_age_c;
  int beta1_gt_0;
  int beta2_lt_0;

  peak_age_c = -beta1 / (2 * beta2);
  beta1_gt_0 = beta1 > 0 ? 1 : 0;
  beta2_lt_0 = beta2 < 0 ? 1 : 0;
```

```
}
"

writeLines(simple_stan_code, "simple_model.stan")
mod_simple <- cmdstan_model("simple_model.stan")
```

#fit HMC model 1

```
fit_simple_hmc <- mod_simple$sample(
  seed = 1,
  refresh = 500,
  chains = 4,
  iter_warmup = 1000,
  iter_sampling = 2000,
  data = stan_data_simple
)
```

```
## Running MCMC with 4 sequential chains...
##
## Chain 1 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 1 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 1 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 1 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 1 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 1 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 1 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 1 finished in 1.5 seconds.
## Chain 2 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 2 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 2 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 2 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 2 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 2 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 2 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 2 finished in 1.4 seconds.
## Chain 3 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 3 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 3 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 3 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 3 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 3 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 3 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 3 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 3 finished in 1.6 seconds.
## Chain 4 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 4 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 4 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 4 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 4 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 4 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 4 Iteration: 3000 / 3000 [100%]  (Sampling)
```

```
## Chain 4 finished in 1.5 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 1.5 seconds.
## Total execution time: 6.3 seconds.
```

#fit VI

```
fit_simple_vi_meanfield <- mod_simple$variational(
  seed = 1,
  refresh = 500,
  algorithm = "meanfield",
  output_samples = 1000,
  data = stan_data_simple
)
```

```
## ------------------------------------------------------------
## EXPERIMENTAL ALGORITHM:
##   This procedure has not been thoroughly tested and may be unstable
##   or buggy. The interface is subject to change.
## ------------------------------------------------------------
## Gradient evaluation took 0.000116 seconds
## 1000 transitions using 10 leapfrog steps per transition would take 1.16 seconds.
## Adjust your expectations accordingly!
## Begin eta adaptation.
## Iteration:   1 / 250 [  0%]  (Adaptation)
## Iteration:  50 / 250 [ 20%]  (Adaptation)
## Iteration: 100 / 250 [ 40%]  (Adaptation)
## Iteration: 150 / 250 [ 60%]  (Adaptation)
## Iteration: 200 / 250 [ 80%]  (Adaptation)
## Success! Found best value [eta = 1] earlier than expected.
## Begin stochastic gradient ascent.
##   iter             ELBO   delta_ELBO_mean   delta_ELBO_med   notes
##    100       -12810.106             1.000            1.000
##    200       -10580.938             0.605            1.000
##    300       -11170.483             0.421            0.211
##    400       -10595.634             0.329            0.211
##    500       -10679.109             0.265            0.054
##    600       -10790.196             0.223            0.054
##    700       -10811.445             0.191            0.053
##    800       -10744.375             0.168            0.053
##    900       -10947.177             0.151            0.019
##   1000       -10668.063             0.139            0.026
##   1100       -10717.229             0.039            0.019
##   1200       -10681.626             0.019            0.010
##   1300       -10568.100             0.014            0.010
##   1400       -10629.727             0.010            0.008   MEAN ELBO CONVERGED   MEDIAN ELBO CONVE
## Drawing a sample of size 1000 from the approximate posterior...
## COMPLETED.
## Finished in  0.4 seconds.
```

```
fit_simple_vi_fullrank <- mod_simple$variational(
  seed = 1,
  refresh = 500,
  algorithm = "fullrank",
  output_samples = 1000,
```

```
  data = stan_data_simple
)
```

```
## ------------------------------------------------------------
## EXPERIMENTAL ALGORITHM:
##   This procedure has not been thoroughly tested and may be unstable
##   or buggy. The interface is subject to change.
## ------------------------------------------------------------
## Gradient evaluation took 0.000134 seconds
## 1000 transitions using 10 leapfrog steps per transition would take 1.34 seconds.
## Adjust your expectations accordingly!
## Begin eta adaptation.
## Iteration:   1 / 250 [  0%]  (Adaptation)
## Iteration:  50 / 250 [ 20%]  (Adaptation)
## Iteration: 100 / 250 [ 40%]  (Adaptation)
## Iteration: 150 / 250 [ 60%]  (Adaptation)
## Iteration: 200 / 250 [ 80%]  (Adaptation)
## Success! Found best value [eta = 1] earlier than expected.
## Begin stochastic gradient ascent.
##   iter             ELBO   delta_ELBO_mean   delta_ELBO_med   notes
##    100       -44914.463             1.000            1.000
##    200       -20542.670             1.093            1.186
##    300       -13469.307             0.904            1.000
##    400       -11776.560             0.714            1.000
##    500       -11526.818             0.575            0.525
##    600       -11101.411             0.486            0.525
##    700       -16937.895             0.466            0.345
##    800       -11447.266             0.467            0.480
##    900       -11678.645             0.418            0.345
##   1000       -12392.836             0.382            0.345
##   1100       -12796.364             0.285            0.144
##   1200       -12865.075             0.167            0.058
##   1300       -15563.003             0.132            0.058
##   1400       -11424.680             0.153            0.058
##   1500       -10792.390             0.157            0.059
##   1600       -10888.794             0.154            0.059
##   1700       -13976.924             0.142            0.059
##   1800       -11345.661             0.117            0.059
##   1900       -10788.964             0.120            0.059
##   2000       -10767.044             0.115            0.059
##   2100       -10610.493             0.113            0.059
##   2200       -10682.144             0.113            0.059
##   2300       -10652.410             0.096            0.052
##   2400       -11021.246             0.063            0.033
##   2500       -11141.596             0.058            0.015
##   2600       -11141.997             0.058            0.015
##   2700       -10915.934             0.037            0.015
##   2800       -10802.886             0.015            0.011
##   2900       -11557.409             0.017            0.011
##   3000       -10649.615             0.025            0.015
##   3100       -11298.268             0.029            0.021
##   3200       -12223.483             0.036            0.033
##   3300       -10918.915             0.048            0.057
##   3400       -11346.550             0.048            0.057
```

```
##   3500       -11438.023             0.048            0.057
##   3600       -10706.753             0.055            0.065
##   3700       -10625.608             0.054            0.065
##   3800       -10768.627             0.054            0.065
##   3900       -10749.325             0.047            0.057
##   4000       -10882.881             0.040            0.038
##   4100       -10780.048             0.035            0.013
##   4200       -10636.070             0.029            0.013
##   4300       -10706.241             0.018            0.012
##   4400       -10795.912             0.015            0.010   MEDIAN ELBO CONVERGED
## Drawing a sample of size 1000 from the approximate posterior...
## COMPLETED.
## Finished in  0.9 seconds.
```

#model1 laplace

```
fit_simple_laplace <- mod_simple$laplace(
  seed = 1,
  data = stan_data_simple
)
```

```
## Initial log joint probability = -1.05528e+06
##     Iter      log prob        ||dx||      ||grad||       alpha      alpha0  # evals  Notes
##        9       -633358   0.000209174        14.813           1           1       14
## Optimization terminated normally:
##   Convergence detected: relative gradient magnitude is below tolerance
## Finished in  0.1 seconds.
## Calculating Hessian
## Calculating inverse of Cholesky factor
## Generating draws
## iteration: 0
## iteration: 100
## iteration: 200
## iteration: 300
## iteration: 400
## iteration: 500
## iteration: 600
## iteration: 700
## iteration: 800
## iteration: 900
## Finished in  0.1 seconds.
```

#check 4 models

```
fit_simple_hmc$summary(c("beta0", "beta1", "beta2", "peak_age_c"))
```

```
## # A tibble: 4 x 10
##   variable      mean  median      sd     mad      q5     q95  rhat ess_bulk
##   <chr>        <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl>    <dbl>
## 1 beta0      -0.960  -0.960  0.00278 0.00280 -0.965  -0.955   1.00    3949.
## 2 beta1      -0.0271 -0.0271 0.00231 0.00231 -0.0308 -0.0232  1.00    4871.
## 3 beta2      -0.0165 -0.0165 0.00201 0.00203 -0.0199 -0.0132  1.00    4644.
## 4 peak_age_c -0.832  -0.820  0.131   0.123   -1.06   -0.645   1.00    4450.
## # i 1 more variable: ess_tail <dbl>
```

```
fit_simple_vi_meanfield$summary(c("beta0", "beta1", "beta2", "peak_age_c"))
```

```
## # A tibble: 4 x 7
##   variable      mean  median      sd     mad      q5     q95
##   <chr>        <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 beta0      -0.971  -0.971  0.00232 0.00224 -0.974  -0.967
## 2 beta1      -0.0192 -0.0192 0.00181 0.00188 -0.0221 -0.0162
## 3 beta2      -0.0283 -0.0283 0.00184 0.00182 -0.0313 -0.0253
## 4 peak_age_c -0.340  -0.338  0.0393  0.0394  -0.407  -0.274
```

```
fit_simple_vi_fullrank$summary(c("beta0", "beta1", "beta2", "peak_age_c"))
```

```
## # A tibble: 4 x 7
##   variable      mean  median       sd     mad      q5     q95
##   <chr>        <dbl>   <dbl>    <dbl>   <dbl>   <dbl>   <dbl>
## 1 beta0      -0.949  -0.949   0.00574 0.00559 -0.958  -0.939
## 2 beta1      -0.0281 -0.0281  0.00701 0.00702 -0.0399 -0.0169
## 3 beta2      -0.0265 -0.0268  0.0297  0.0295  -0.0754  0.0232
## 4 peak_age_c -1.11   -0.341  42.1     0.318   -2.65    1.93
```

```
fit_simple_laplace$summary(c("beta0", "beta1", "beta2", "peak_age_c"))
```

```
## # A tibble: 4 x 7
##   variable      mean  median      sd     mad      q5     q95
##   <chr>        <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 beta0      -0.960  -0.960  0.00284 0.00284 -0.965  -0.955
## 2 beta1      -0.0270 -0.0270 0.00224 0.00231 -0.0306 -0.0234
## 3 beta2      -0.0165 -0.0166 0.00199 0.00202 -0.0196 -0.0131
## 4 peak_age_c -0.831  -0.819  0.128   0.123   -1.07   -0.646
```

#posterior draws

```
draws_simple_hmc <- fit_simple_hmc$draws(
  variables = c("beta0", "beta1", "beta2", "peak_age_c", "beta1_gt_0", "beta2_lt_0"),
  format = "df"
)

draws_simple_vi_meanfield <- fit_simple_vi_meanfield$draws(
  variables = c("beta0", "beta1", "beta2", "peak_age_c", "beta1_gt_0", "beta2_lt_0"),
  format = "df"
)

draws_simple_vi_fullrank <- fit_simple_vi_fullrank$draws(
  variables = c("beta0", "beta1", "beta2", "peak_age_c", "beta1_gt_0", "beta2_lt_0"),
  format = "df"
)

draws_simple_laplace <- fit_simple_laplace$draws(
  variables = c("beta0", "beta1", "beta2", "peak_age_c", "beta1_gt_0", "beta2_lt_0"),
  format = "df"
)
```

#Convert peak age back to the original age scale

```
draws_simple_hmc$peak_age <- age_center + age_scale * draws_simple_hmc$peak_age_c
draws_simple_vi_meanfield$peak_age <- age_center + age_scale * draws_simple_vi_meanfield$peak_age_c
draws_simple_vi_fullrank$peak_age <- age_center + age_scale * draws_simple_vi_fullrank$peak_age_c
draws_simple_laplace$peak_age <- age_center + age_scale * draws_simple_laplace$peak_age_c
```

compare posterior

```
compare_simple_table <- data.frame(
  method = c("HMC", "VI_meanfield", "VI_fullrank", "Laplace"),

  beta1_mean = c(
    mean(draws_simple_hmc$beta1),
    mean(draws_simple_vi_meanfield$beta1),
    mean(draws_simple_vi_fullrank$beta1),
    mean(draws_simple_laplace$beta1)
  ),
  beta1_low = c(
    quantile(draws_simple_hmc$beta1, 0.025),
    quantile(draws_simple_vi_meanfield$beta1, 0.025),
    quantile(draws_simple_vi_fullrank$beta1, 0.025),
    quantile(draws_simple_laplace$beta1, 0.025)
  ),
  beta1_high = c(
    quantile(draws_simple_hmc$beta1, 0.975),
    quantile(draws_simple_vi_meanfield$beta1, 0.975),
    quantile(draws_simple_vi_fullrank$beta1, 0.975),
    quantile(draws_simple_laplace$beta1, 0.975)
  ),

  beta2_mean = c(
    mean(draws_simple_hmc$beta2),
    mean(draws_simple_vi_meanfield$beta2),
    mean(draws_simple_vi_fullrank$beta2),
    mean(draws_simple_laplace$beta2)
  ),
  beta2_low = c(
    quantile(draws_simple_hmc$beta2, 0.025),
    quantile(draws_simple_vi_meanfield$beta2, 0.025),
    quantile(draws_simple_vi_fullrank$beta2, 0.025),
    quantile(draws_simple_laplace$beta2, 0.025)
  ),
  beta2_high = c(
    quantile(draws_simple_hmc$beta2, 0.975),
    quantile(draws_simple_vi_meanfield$beta2, 0.975),
    quantile(draws_simple_vi_fullrank$beta2, 0.975),
    quantile(draws_simple_laplace$beta2, 0.975)
  ),

  peak_mean = c(
    mean(draws_simple_hmc$peak_age),
    mean(draws_simple_vi_meanfield$peak_age),
    mean(draws_simple_vi_fullrank$peak_age),
    mean(draws_simple_laplace$peak_age)
  ),
  peak_low = c(
    quantile(draws_simple_hmc$peak_age, 0.025),
    quantile(draws_simple_vi_meanfield$peak_age, 0.025),
    quantile(draws_simple_vi_fullrank$peak_age, 0.025),
```

```
    quantile(draws_simple_laplace$peak_age, 0.025)
  ),
  peak_high = c(
    quantile(draws_simple_hmc$peak_age, 0.975),
    quantile(draws_simple_vi_meanfield$peak_age, 0.975),
    quantile(draws_simple_vi_fullrank$peak_age, 0.975),
    quantile(draws_simple_laplace$peak_age, 0.975)
  ),

  prob_beta1_gt_0 = c(
    mean(draws_simple_hmc$beta1_gt_0),
    mean(draws_simple_vi_meanfield$beta1_gt_0),
    mean(draws_simple_vi_fullrank$beta1_gt_0),
    mean(draws_simple_laplace$beta1_gt_0)
  ),
  prob_beta2_lt_0 = c(
    mean(draws_simple_hmc$beta2_lt_0),
    mean(draws_simple_vi_meanfield$beta2_lt_0),
    mean(draws_simple_vi_fullrank$beta2_lt_0),
    mean(draws_simple_laplace$beta2_lt_0)
  )
)

compare_simple_table
```

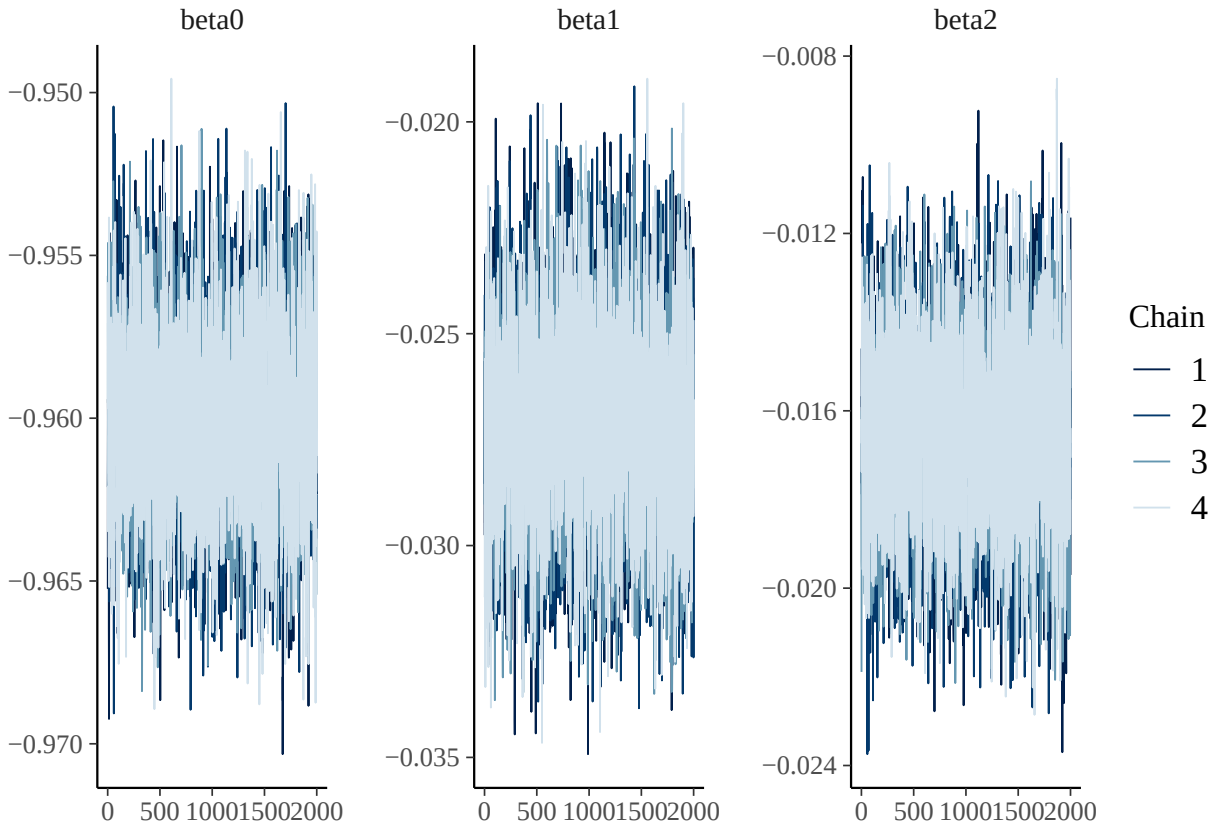
```
##         method  beta1_mean   beta1_low  beta1_high  beta2_mean   beta2_low
## 1          HMC -0.02708542 -0.03159674 -0.02249717 -0.01653139 -0.02045139
## 2 VI_meanfield -0.01916449 -0.02260964 -0.01566982 -0.02829465 -0.03175494
## 3  VI_fullrank -0.02812354 -0.04223340 -0.01503988 -0.02647058 -0.08521472
## 4      Laplace -0.02700420 -0.03127925 -0.02274087 -0.01650745 -0.02021578
##    beta2_high peak_mean  peak_low peak_high prob_beta1_gt_0 prob_beta2_lt_0
## 1 -0.01260642  26.42664 25.159045  27.40485               0           1.000
## 2 -0.02469528  28.59258 28.233671  28.92010               0           1.000
## 3  0.03179994  25.21857  7.589677  45.69322               0           0.814
## 4 -0.01262256  26.43405 25.142013  27.34608               0           1.000
```

#HMC diagnostic model 1

```
fit_simple_hmc$diagnostic_summary()
```

```
## $num_divergent
## [1] 0 0 0 0
##
## $num_max_treedepth
## [1] 0 0 0 0
##
## $ebfmi
## [1] 1.090644 1.111539 1.075261 1.094578
```

```
mcmc_trace(
  as_draws_array(fit_simple_hmc$draws(c("beta0", "beta1", "beta2")))
)
```

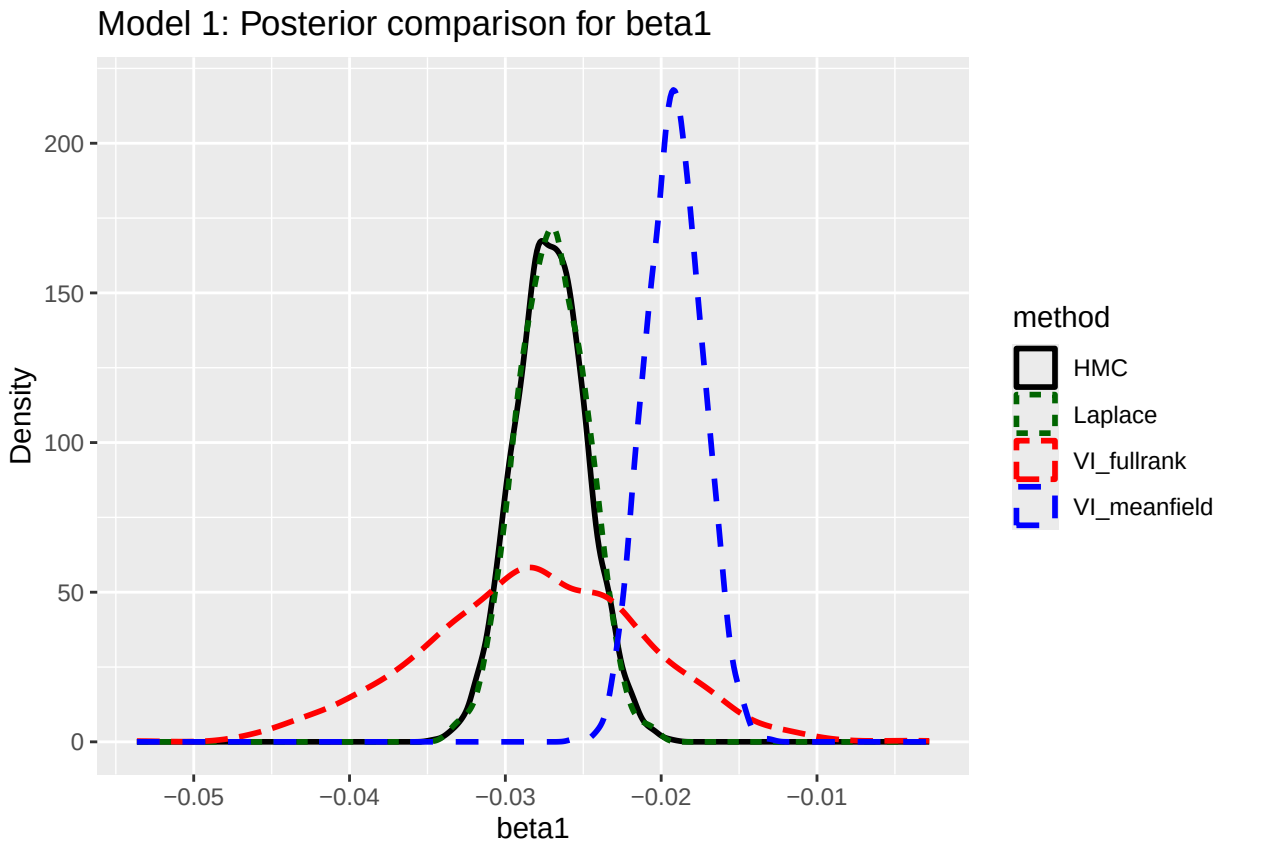


#plots for HMC

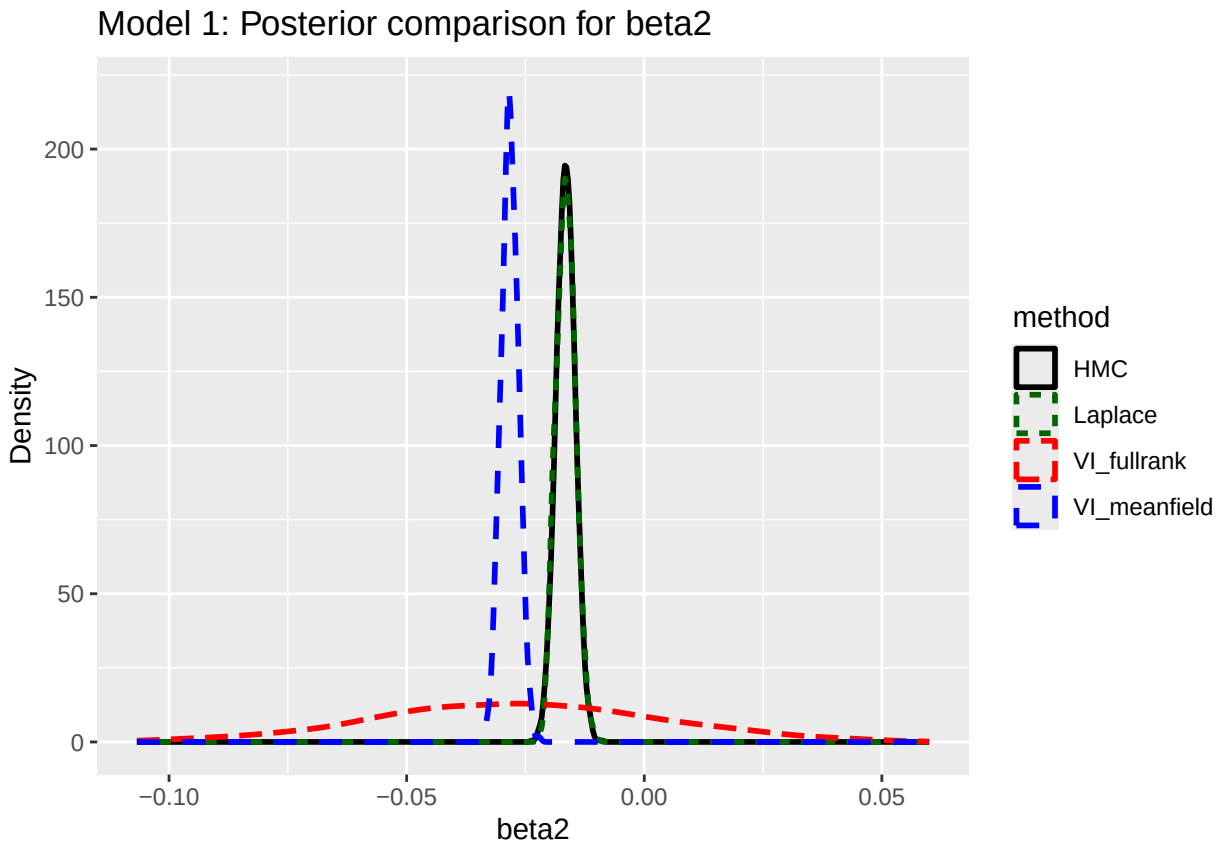
```
method_colors <- c(
  "HMC" = "black",
  "VI_fullrank" = "red",
  "VI_meanfield" = "blue",
  "Laplace" = "darkgreen"
)

# beta1
plot_df_beta1_simple <- data.frame(
  value = c(
    draws_simple_hmc$beta1,
    draws_simple_vi_meanfield$beta1,
    draws_simple_vi_fullrank$beta1,
    draws_simple_laplace$beta1
  ),
  method = c(
    rep("HMC", nrow(draws_simple_hmc)),
    rep("VI_meanfield", nrow(draws_simple_vi_meanfield)),
    rep("VI_fullrank", nrow(draws_simple_vi_fullrank)),
    rep("Laplace", nrow(draws_simple_laplace))
  )
)

p_beta1 <- ggplot(plot_df_beta1_simple, aes(x = value, color = method, linetype = method)) +
  geom_density(linewidth = 1) +
  scale_color_manual(values = method_colors) +
```

```
  labs(
    title = "Model 1: Posterior comparison for beta1",
    x = "beta1",
    y = "Density"
  )

# beta2
plot_df_beta2_simple <- data.frame(
  value = c(
    draws_simple_hmc$beta2,
    draws_simple_vi_meanfield$beta2,
    draws_simple_vi_fullrank$beta2,
    draws_simple_laplace$beta2
  ),
  method = c(
    rep("HMC", nrow(draws_simple_hmc)),
    rep("VI_meanfield", nrow(draws_simple_vi_meanfield)),
    rep("VI_fullrank", nrow(draws_simple_vi_fullrank)),
    rep("Laplace", nrow(draws_simple_laplace))
  )
)

p_beta2 <- ggplot(plot_df_beta2_simple, aes(x = value, color = method, linetype = method)) +
  geom_density(linewidth = 1) +
  scale_color_manual(values = method_colors) +
  labs(
    title = "Model 1: Posterior comparison for beta2",
    x = "beta2",
    y = "Density"
  )

# peak age
plot_df_peak_simple <- data.frame(
  value = c(
    draws_simple_hmc$peak_age,
    draws_simple_vi_meanfield$peak_age,
    draws_simple_vi_fullrank$peak_age,
    draws_simple_laplace$peak_age
  ),
  method = c(
    rep("HMC", nrow(draws_simple_hmc)),
    rep("VI_meanfield", nrow(draws_simple_vi_meanfield)),
    rep("VI_fullrank", nrow(draws_simple_vi_fullrank)),
    rep("Laplace", nrow(draws_simple_laplace))
  )
)

plot_df_peak_simple <- data.frame(
  value = c(
    draws_simple_hmc$peak_age,
    draws_simple_vi_meanfield$peak_age,
    draws_simple_vi_fullrank$peak_age,
    draws_simple_laplace$peak_age
```

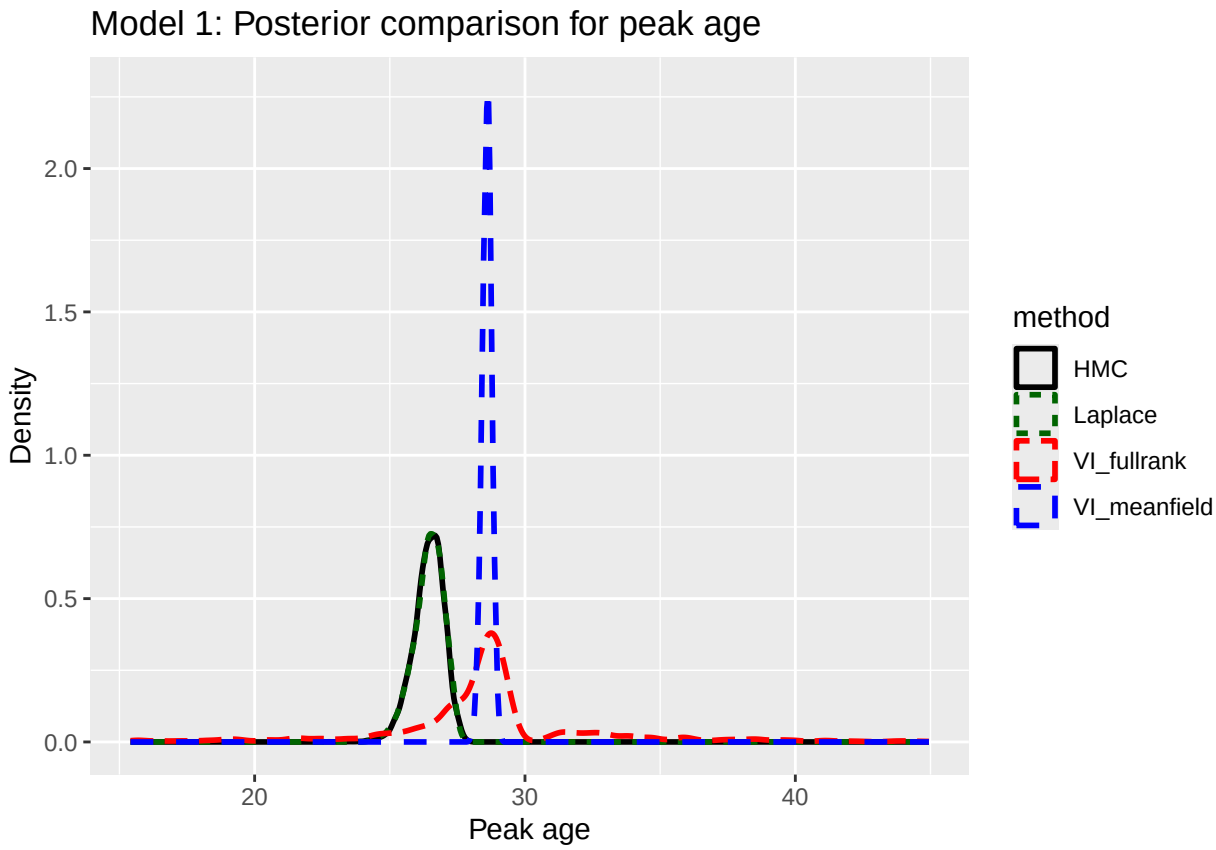
```
  ),
  method = c(
    rep("HMC", nrow(draws_simple_hmc)),
    rep("VI_meanfield", nrow(draws_simple_vi_meanfield)),
    rep("VI_fullrank", nrow(draws_simple_vi_fullrank)),
    rep("Laplace", nrow(draws_simple_laplace))
  )
)

plot_df_peak_simple <- subset(plot_df_peak_simple, value >= 15 & value <= 45)

p_peak <- ggplot(plot_df_peak_simple, aes(x = value, color = method, linetype = method)) +
  geom_density(linewidth = 1) +
  scale_color_manual(values = method_colors) +
  labs(
    title = "Model 1: Posterior comparison for peak age",
    x = "Peak age",
    y = "Density"
  )


# show plots
p_beta1
```




```
p_beta2
```


Model 1: Posterior comparison for beta2
Density
200
150
100
50
0
−0.10
−0.05
0.00
0.05
beta2
method
HMC
Laplace
VI_fullrank
VI_meanfield

```
p_peak
```



model 2

```
hier_stan_code <- "
data {
  int<lower=1> N;
  int<lower=1> J;
  array[N] int<lower=1, upper=J> player;
  array[N] int<lower=0> AB;
  array[N] int<lower=0> H;
  vector[N] age_c;
  vector[N] age_c2;
}

parameters {
  real mu_alpha;
  real<lower=0> sigma_alpha;
  vector[J] alpha_raw;
  real beta1;
  real beta2;
}

transformed parameters {
  vector[J] alpha;
  vector[N] logit_theta;
```

```
  alpha = mu_alpha + sigma_alpha * alpha_raw;

  for (n in 1:N) {
    logit_theta[n] = alpha[player[n]] + beta1 * age_c[n] + beta2 * age_c2[n];
  }
}

model {
  mu_alpha ~ normal(0, 5);
  sigma_alpha ~ exponential(1);
  alpha_raw ~ normal(0, 1);
  beta1 ~ normal(0, 5);
  beta2 ~ normal(0, 5);

  H ~ binomial_logit(AB, logit_theta);
}

generated quantities {
  real peak_age_c;
  int beta1_gt_0;
  int beta2_lt_0;

  peak_age_c = -beta1 / (2 * beta2);
  beta1_gt_0 = beta1 > 0 ? 1 : 0;
  beta2_lt_0 = beta2 < 0 ? 1 : 0;
}
"

writeLines(hier_stan_code, "hier_model.stan")
mod_hier <- cmdstan_model("hier_model.stan")
```

HMC model 2

```
fit_hier_hmc <- mod_hier$sample(
  seed = 1,
  refresh = 500,
  chains = 4,
  iter_warmup = 1000,
  iter_sampling = 2000,
  data = stan_data_hier
)
```

```
## Running MCMC with 4 sequential chains...
##
## Chain 1 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 1 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 1 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 1 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 1 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 1 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 1 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 1 finished in 13.3 seconds.
```

```
## Chain 2 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 2 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 2 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 2 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 2 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 2 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 2 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 2 finished in 13.1 seconds.
## Chain 3 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 3 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 3 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 3 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 3 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 3 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 3 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 3 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 3 finished in 13.4 seconds.
## Chain 4 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 4 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 4 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 4 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 4 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 4 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 4 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 4 finished in 12.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 13.0 seconds.
## Total execution time: 52.4 seconds.
```

VI model 2

```
fit_hier_vi_meanfield <- mod_hier$variational(
  seed = 1,
  refresh = 500,
  algorithm = "meanfield",
  output_samples = 1000,
  data = stan_data_hier
)
```

```
## ------------------------------------------------------------
## EXPERIMENTAL ALGORITHM:
##   This procedure has not been thoroughly tested and may be unstable
##   or buggy. The interface is subject to change.
## ------------------------------------------------------------
## Gradient evaluation took 0.000157 seconds
## 1000 transitions using 10 leapfrog steps per transition would take 1.57 seconds.
## Adjust your expectations accordingly!
## Begin eta adaptation.
## Iteration:   1 / 250 [  0%]  (Adaptation)
## Iteration:  50 / 250 [ 20%]  (Adaptation)
```

```
## Iteration: 100 / 250 [ 40%]  (Adaptation)
## Iteration: 150 / 250 [ 60%]  (Adaptation)
## Iteration: 200 / 250 [ 80%]  (Adaptation)
## Success! Found best value [eta = 1] earlier than expected.
## Begin stochastic gradient ascent.
##   iter             ELBO   delta_ELBO_mean   delta_ELBO_med   notes
##    100       -11678.678             1.000            1.000
##    200       -10248.688             0.570            1.000
##    300        -9873.330             0.393            0.140
##    400       -10021.865             0.298            0.140
##    500        -9965.060             0.240            0.038
##    600       -10132.102             0.202            0.038
##    700        -9917.674             0.177            0.022
##    800       -10182.826             0.158            0.026
##    900       -10293.502             0.141            0.022
##   1000        -9902.598             0.131            0.026
##   1100        -9878.808             0.031            0.022
##   1200        -9907.465             0.018            0.016
##   1300       -10014.588             0.015            0.015
##   1400       -10054.019             0.014            0.011
##   1500        -9943.865             0.015            0.011
##   1600        -9910.283             0.013            0.011
##   1700        -9860.343             0.012            0.011
##   1800        -9899.761             0.009            0.005   MEAN ELBO CONVERGED   MEDIAN ELBO CONVE
## Drawing a sample of size 1000 from the approximate posterior...
## COMPLETED.
## Finished in  1.1 seconds.
```

```
fit_hier_vi_fullrank <- mod_hier$variational(
  seed = 1,
  refresh = 500,
  algorithm = "fullrank",
  output_samples = 1000,
  data = stan_data_hier
)
```

```
## ------------------------------------------------------------
## EXPERIMENTAL ALGORITHM:
##   This procedure has not been thoroughly tested and may be unstable
##   or buggy. The interface is subject to change.
## ------------------------------------------------------------
## Gradient evaluation took 0.000191 seconds
## 1000 transitions using 10 leapfrog steps per transition would take 1.91 seconds.
## Adjust your expectations accordingly!
## Begin eta adaptation.
## Iteration:   1 / 250 [  0%]  (Adaptation)
## Iteration:  50 / 250 [ 20%]  (Adaptation)
## Iteration: 100 / 250 [ 40%]  (Adaptation)
## Iteration: 150 / 250 [ 60%]  (Adaptation)
## Iteration: 200 / 250 [ 80%]  (Adaptation)
## Iteration: 250 / 250 [100%]  (Adaptation)
## Success! Found best value [eta = 0.1].
## Begin stochastic gradient ascent.
##   iter             ELBO   delta_ELBO_mean   delta_ELBO_med   notes
##    100     -1140770.177             1.000            1.000
```

```
##    200      -788486.959           0.723           1.000
##    300      -702882.131           0.523           0.447
##    400      -503077.672           0.491           0.447
##    500      -516343.934           0.398           0.397
##    600      -369842.519           0.398           0.397
##    700      -310008.814           0.369           0.396
##    800      -268368.795           0.342           0.396
##    900      -210338.909           0.335           0.276
##   1000      -162874.983           0.330           0.291
##   1100      -125808.711           0.260           0.291
##   1200      -102764.279           0.238           0.276
##   1300       -74854.920           0.263           0.291
##   1400       -52114.566           0.267           0.291
##   1500       -58264.047           0.275           0.291
##   1600       -45342.846           0.263           0.285
##   1700       -37478.941           0.265           0.285
##   1800       -34117.033           0.259           0.285
##   1900       -28879.670           0.250           0.285
##   2000       -29293.138           0.222           0.224
##   2100       -27520.514           0.199           0.210
##   2200       -24925.552           0.187           0.181
##   2300       -25133.831           0.151           0.106
##   2400       -22425.107           0.119           0.106
##   2500       -21442.518           0.113           0.104
##   2600       -19649.933           0.094           0.099
##   2700       -21352.283           0.081           0.091
##   2800       -20592.620           0.075           0.080
##   2900       -22477.685           0.065           0.080
##   3000       -18855.848           0.083           0.084
##   3100       -19198.977           0.078           0.084
##   3200       -19187.449           0.068           0.080
##   3300       -17902.344           0.074           0.080
##   3400       -19364.700           0.070           0.076
##   3500       -17843.954           0.073           0.080
##   3600       -18546.810           0.068           0.076
##   3700       -18312.927           0.061           0.072
##   3800       -20471.273           0.068           0.076
##   3900       -17937.038           0.074           0.076
##   4000       -17839.730           0.055           0.072
##   4100       -16653.418           0.061           0.072
##   4200       -18493.844           0.071           0.076
##   4300       -16251.693           0.077           0.085
##   4400       -17714.421           0.078           0.085
##   4500       -16743.658           0.075           0.083
##   4600       -17268.110           0.074           0.083
##   4700       -16254.611           0.079           0.083
##   4800       -16435.064           0.070           0.071
##   4900       -15941.042           0.059           0.062
##   5000       -15423.156           0.062           0.062
##   5100       -16380.445           0.060           0.058
##   5200       -16035.348           0.053           0.058
##   5300       -15813.995           0.040           0.034
##   5400       -17260.658           0.040           0.034
##   5500       -15198.082           0.048           0.034
```

```
##   5600       -14960.019            0.047            0.034
##   5700       -15662.171            0.045            0.034
##   5800       -15863.915            0.045            0.034
##   5900       -15876.126            0.042            0.034
##   6000       -15625.999            0.040            0.022
##   6100       -15347.880            0.036            0.018
##   6200       -14446.446            0.040            0.018
##   6300       -14692.935            0.041            0.018
##   6400       -14672.324            0.032            0.017
##   6500       -14229.915            0.022            0.017
##   6600       -15074.783            0.026            0.018
##   6700       -13903.796            0.030            0.018
##   6800       -14997.306            0.036            0.031
##   6900       -14614.483            0.039            0.031
##   7000       -16033.842            0.046            0.056
##   7100       -13973.105            0.059            0.062
##   7200       -14268.394            0.055            0.056
##   7300       -14403.942            0.054            0.056
##   7400       -13849.755            0.058            0.056
##   7500       -14435.144            0.059            0.056
##   7600       -13413.582            0.061            0.073
##   7700       -14144.146            0.057            0.052
##   7800       -13842.684            0.052            0.041
##   7900       -13411.853            0.053            0.041
##   8000       -13514.748            0.045            0.040
##   8100       -13573.367            0.030            0.032
##   8200       -13668.025            0.029            0.032
##   8300       -13425.573            0.030            0.032
##   8400       -14552.904            0.034            0.032
##   8500       -13441.364            0.038            0.032
##   8600       -12961.010            0.034            0.032
##   8700       -13303.113            0.031            0.026
##   8800       -13022.922            0.031            0.026
##   8900       -13449.984            0.031            0.026
##   9000       -13964.800            0.034            0.032
##   9100       -13922.065            0.034            0.032
##   9200       -13291.531            0.038            0.037
##   9300       -12342.207            0.044            0.037
##   9400       -12939.635            0.041            0.037
##   9500       -13130.494            0.034            0.037
##   9600       -13487.584            0.033            0.032
##   9700       -12840.713            0.036            0.037
##   9800       -13167.387            0.036            0.037
##   9900       -12639.752            0.037            0.042
##   10000       -12804.376            0.034            0.042
## Informational Message: The maximum number of iterations is reached! The algorithm may not have conve
## This variational approximation is not guaranteed to be meaningful.
## Drawing a sample of size 1000 from the approximate posterior...
## COMPLETED.
## Finished in  4.9 seconds.
```

Laplace with Gaussian approximation

```
log_post_hier <- function(par, data, J) {
  mu_alpha <- par[1]
  beta1 <- par[2]
  beta2 <- par[3]
  log_sigma <- par[4]
  alpha <- par[5:(4 + J)]

  sigma_alpha <- exp(log_sigma)

  eta <- alpha[data$player_id] + beta1 * data$age_c + beta2 * data$age_c2
  p <- plogis(eta)

  loglik <- sum(dbinom(data$H, size = data$AB, prob = p, log = TRUE))

  logprior_alpha <- sum(dnorm(alpha, mean = mu_alpha, sd = sigma_alpha, log = TRUE))

  logprior_hyper <-
    dnorm(mu_alpha, mean = 0, sd = 5, log = TRUE) +
    dnorm(beta1, mean = 0, sd = 5, log = TRUE) +
    dnorm(beta2, mean = 0, sd = 5, log = TRUE) +
    dexp(sigma_alpha, rate = 1, log = TRUE) +
    log_sigma

  out <- loglik + logprior_alpha + logprior_hyper

  if (!is.finite(out)) return(-1e12)
  out
}

neg_log_post_hier <- function(par, data, J) {
  -log_post_hier(par, data, J)
}

overall_rate <- sum(data$H) / sum(data$AB)
overall_rate <- pmin(pmax(overall_rate, 1e-6), 1 - 1e-6)

init_mu <- qlogis(overall_rate)

init_par <- c(
  mu_alpha = init_mu,
  beta1 = 0,
  beta2 = 0,
  log_sigma = log(0.5),
  alpha = rep(init_mu, J)
)

fit_map_hier <- optim(
  par = init_par,
  fn = neg_log_post_hier,
  data = data,
  J = J,
```

```
  method = "BFGS",
  hessian = TRUE,
  control = list(maxit = 2000, reltol = 1e-10)
)

map_est_hier <- fit_map_hier$par

mu_vec <- map_est_hier[1:4]

H_top <- fit_map_hier$hessian[1:4, 1:4]
H_top <- (H_top + t(H_top)) / 2

eps <- 1e-6
Sigma_mat <- MASS::ginv(H_top + eps * diag(4))
Sigma_mat <- (Sigma_mat + t(Sigma_mat)) / 2

set.seed(123)
n_draws <- 10000

draw_mat <- MASS::mvrnorm(
  n = n_draws,
  mu = mu_vec,
  Sigma = Sigma_mat
)

draws_hier_laplace <- data.frame(
  mu_alpha = draw_mat[, 1],
  beta1 = draw_mat[, 2],
  beta2 = draw_mat[, 3],
  log_sigma = draw_mat[, 4]
)

draws_hier_laplace$sigma_alpha <- exp(draws_hier_laplace$log_sigma)
draws_hier_laplace$peak_age_c <- -draws_hier_laplace$beta1 / (2 * draws_hier_laplace$beta2)
draws_hier_laplace$beta1_gt_0 <- ifelse(draws_hier_laplace$beta1 > 0, 1, 0)
draws_hier_laplace$beta2_lt_0 <- ifelse(draws_hier_laplace$beta2 < 0, 1, 0)
draws_hier_laplace$peak_age <- age_center + age_scale * draws_hier_laplace$peak_age_c
```

```
fit_hier_hmc$summary(c("mu_alpha", "sigma_alpha", "beta1", "beta2", "peak_age_c"))
```

```
## # A tibble: 5 x 10
##   variable       mean  median      sd     mad      q5     q95  rhat ess_bulk
##   <chr>         <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl>    <dbl>
## 1 mu_alpha    -0.972  -0.972  0.00755 0.00762 -0.984  -0.959  1.00      949.
## 2 sigma_alpha  0.0975  0.0973 0.00562 0.00564  0.0886  0.107  1.00     1127.
## 3 beta1       -0.0374 -0.0374 0.00250 0.00249 -0.0415 -0.0333 1.00    12071.
## 4 beta2       -0.0282 -0.0282 0.00212 0.00217 -0.0317 -0.0247 1.000   13568.
## 5 peak_age_c  -0.666  -0.663  0.0692  0.0678  -0.787  -0.559  1.000   13491.
## # i 1 more variable: ess_tail <dbl>
```

```
fit_hier_vi_meanfield$summary(c("mu_alpha", "sigma_alpha", "beta1", "beta2", "peak_age_c"))
```

```
## # A tibble: 5 x 7
##   variable       mean  median      sd     mad      q5     q95
##   <chr>         <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
```

```
## 1 mu_alpha    -0.979  -0.979  0.00149 0.00153 -0.982  -0.977
## 2 sigma_alpha  0.0881  0.0881 0.00232 0.00234  0.0844  0.0919
## 3 beta1       -0.0302 -0.0303 0.00214 0.00219 -0.0336 -0.0266
## 4 beta2       -0.0384 -0.0384 0.00165 0.00175 -0.0411 -0.0357
## 5 peak_age_c  -0.394  -0.393  0.0328  0.0340  -0.447  -0.341
```

```
fit_hier_vi_fullrank$summary(c("mu_alpha", "sigma_alpha", "beta1", "beta2", "peak_age_c"))
```

```
## # A tibble: 5 x 7
##   variable        mean  median        sd     mad      q5     q95
##   <chr>          <dbl>   <dbl>     <dbl>   <dbl>   <dbl>   <dbl>
## 1 mu_alpha     -0.927  -0.927    0.0158  0.0159  -0.954  -0.902
## 2 sigma_alpha   0.0742  0.0742   0.00364 0.00352  0.0686  0.0803
## 3 beta1        -0.0352 -0.0395   0.103   0.0996  -0.205   0.136
## 4 beta2        -0.0475 -0.0461   0.0970  0.0984  -0.212   0.108
## 5 peak_age_c  -17.5    -0.0625 549.      0.762   -4.55    2.97
```

sumamry

```
fit_hier_hmc$summary(c("mu_alpha", "sigma_alpha", "beta1", "beta2", "peak_age_c"))
```

```
## # A tibble: 5 x 10
##   variable       mean  median      sd     mad      q5     q95  rhat ess_bulk
##   <chr>         <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl>    <dbl>
## 1 mu_alpha    -0.972  -0.972  0.00755 0.00762 -0.984  -0.959  1.00      949.
## 2 sigma_alpha  0.0975  0.0973 0.00562 0.00564  0.0886  0.107  1.00     1127.
## 3 beta1       -0.0374 -0.0374 0.00250 0.00249 -0.0415 -0.0333 1.00    12071.
## 4 beta2       -0.0282 -0.0282 0.00212 0.00217 -0.0317 -0.0247 1.000   13568.
## 5 peak_age_c  -0.666  -0.663  0.0692  0.0678  -0.787  -0.559  1.000   13491.
## # i 1 more variable: ess_tail <dbl>
```

```
fit_hier_vi_meanfield$summary(c("mu_alpha", "sigma_alpha", "beta1", "beta2", "peak_age_c"))
```

```
## # A tibble: 5 x 7
##   variable       mean  median      sd     mad      q5     q95
##   <chr>         <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 mu_alpha    -0.979  -0.979  0.00149 0.00153 -0.982  -0.977
## 2 sigma_alpha  0.0881  0.0881 0.00232 0.00234  0.0844  0.0919
## 3 beta1       -0.0302 -0.0303 0.00214 0.00219 -0.0336 -0.0266
## 4 beta2       -0.0384 -0.0384 0.00165 0.00175 -0.0411 -0.0357
## 5 peak_age_c  -0.394  -0.393  0.0328  0.0340  -0.447  -0.341
```

```
fit_hier_vi_fullrank$summary(c("mu_alpha", "sigma_alpha", "beta1", "beta2", "peak_age_c"))
```

```
## # A tibble: 5 x 7
##   variable        mean  median        sd     mad      q5     q95
##   <chr>          <dbl>   <dbl>     <dbl>   <dbl>   <dbl>   <dbl>
## 1 mu_alpha     -0.927  -0.927    0.0158  0.0159  -0.954  -0.902
## 2 sigma_alpha   0.0742  0.0742   0.00364 0.00352  0.0686  0.0803
## 3 beta1        -0.0352 -0.0395   0.103   0.0996  -0.205   0.136
## 4 beta2        -0.0475 -0.0461   0.0970  0.0984  -0.212   0.108
## 5 peak_age_c  -17.5    -0.0625 549.      0.762   -4.55    2.97
```

posterior

```
draws_hier_hmc <- fit_hier_hmc$draws(
  variables = c("mu_alpha", "sigma_alpha", "beta1", "beta2", "peak_age_c", "beta1_gt_0", "beta2_lt_0"),
  format = "df"
)

draws_hier_vi_meanfield <- fit_hier_vi_meanfield$draws(
  variables = c("mu_alpha", "sigma_alpha", "beta1", "beta2", "peak_age_c", "beta1_gt_0", "beta2_lt_0"),
  format = "df"
)

draws_hier_vi_fullrank <- fit_hier_vi_fullrank$draws(
  variables = c("mu_alpha", "sigma_alpha", "beta1", "beta2", "peak_age_c", "beta1_gt_0", "beta2_lt_0"),
  format = "df"
)
```

```
draws_hier_hmc$peak_age <- age_center + age_scale * draws_hier_hmc$peak_age_c
draws_hier_vi_meanfield$peak_age <- age_center + age_scale * draws_hier_vi_meanfield$peak_age_c
draws_hier_vi_fullrank$peak_age <- age_center + age_scale * draws_hier_vi_fullrank$peak_age_c
```

```
compare_hier_table <- data.frame(
  method = c("HMC", "VI_meanfield", "VI_fullrank", "Laplace"),

  beta1_mean = c(
    mean(draws_hier_hmc$beta1),
    mean(draws_hier_vi_meanfield$beta1),
    mean(draws_hier_vi_fullrank$beta1),
    mean(draws_hier_laplace$beta1)
  ),
  beta1_low = c(
    quantile(draws_hier_hmc$beta1, 0.025),
    quantile(draws_hier_vi_meanfield$beta1, 0.025),
    quantile(draws_hier_vi_fullrank$beta1, 0.025),
    quantile(draws_hier_laplace$beta1, 0.025, na.rm = TRUE)
  ),
  beta1_high = c(
    quantile(draws_hier_hmc$beta1, 0.975),
    quantile(draws_hier_vi_meanfield$beta1, 0.975),
    quantile(draws_hier_vi_fullrank$beta1, 0.975),
    quantile(draws_hier_laplace$beta1, 0.975, na.rm = TRUE)
  ),

  beta2_mean = c(
    mean(draws_hier_hmc$beta2),
    mean(draws_hier_vi_meanfield$beta2),
    mean(draws_hier_vi_fullrank$beta2),
    mean(draws_hier_laplace$beta2)
  ),
  beta2_low = c(
    quantile(draws_hier_hmc$beta2, 0.025),
    quantile(draws_hier_vi_meanfield$beta2, 0.025),
    quantile(draws_hier_vi_fullrank$beta2, 0.025),
    quantile(draws_hier_laplace$beta2, 0.025, na.rm = TRUE)
```

```
  ),
  beta2_high = c(
    quantile(draws_hier_hmc$beta2, 0.975),
    quantile(draws_hier_vi_meanfield$beta2, 0.975),
    quantile(draws_hier_vi_fullrank$beta2, 0.975),
    quantile(draws_hier_laplace$beta2, 0.975, na.rm = TRUE)
  ),

  sigma_alpha_mean = c(
    mean(draws_hier_hmc$sigma_alpha),
    mean(draws_hier_vi_meanfield$sigma_alpha),
    mean(draws_hier_vi_fullrank$sigma_alpha),
    mean(draws_hier_laplace$sigma_alpha)
  ),
  sigma_alpha_low = c(
    quantile(draws_hier_hmc$sigma_alpha, 0.025),
    quantile(draws_hier_vi_meanfield$sigma_alpha, 0.025),
    quantile(draws_hier_vi_fullrank$sigma_alpha, 0.025),
    quantile(draws_hier_laplace$sigma_alpha, 0.025, na.rm = TRUE)
  ),
  sigma_alpha_high = c(
    quantile(draws_hier_hmc$sigma_alpha, 0.975),
    quantile(draws_hier_vi_meanfield$sigma_alpha, 0.975),
    quantile(draws_hier_vi_fullrank$sigma_alpha, 0.975),
    quantile(draws_hier_laplace$sigma_alpha, 0.975, na.rm = TRUE)
  ),

  peak_mean = c(
    mean(draws_hier_hmc$peak_age),
    mean(draws_hier_vi_meanfield$peak_age),
    mean(draws_hier_vi_fullrank$peak_age),
    mean(draws_hier_laplace$peak_age, na.rm = TRUE)
  ),
  peak_low = c(
    quantile(draws_hier_hmc$peak_age, 0.025),
    quantile(draws_hier_vi_meanfield$peak_age, 0.025),
    quantile(draws_hier_vi_fullrank$peak_age, 0.025),
    quantile(draws_hier_laplace$peak_age, 0.025, na.rm = TRUE)
  ),
  peak_high = c(
    quantile(draws_hier_hmc$peak_age, 0.975),
    quantile(draws_hier_vi_meanfield$peak_age, 0.975),
    quantile(draws_hier_vi_fullrank$peak_age, 0.975),
    quantile(draws_hier_laplace$peak_age, 0.975, na.rm = TRUE)
  ),

  prob_beta1_gt_0 = c(
    mean(draws_hier_hmc$beta1_gt_0),
    mean(draws_hier_vi_meanfield$beta1_gt_0),
    mean(draws_hier_vi_fullrank$beta1_gt_0),
    mean(draws_hier_laplace$beta1_gt_0)
  ),
  prob_beta2_lt_0 = c(
```

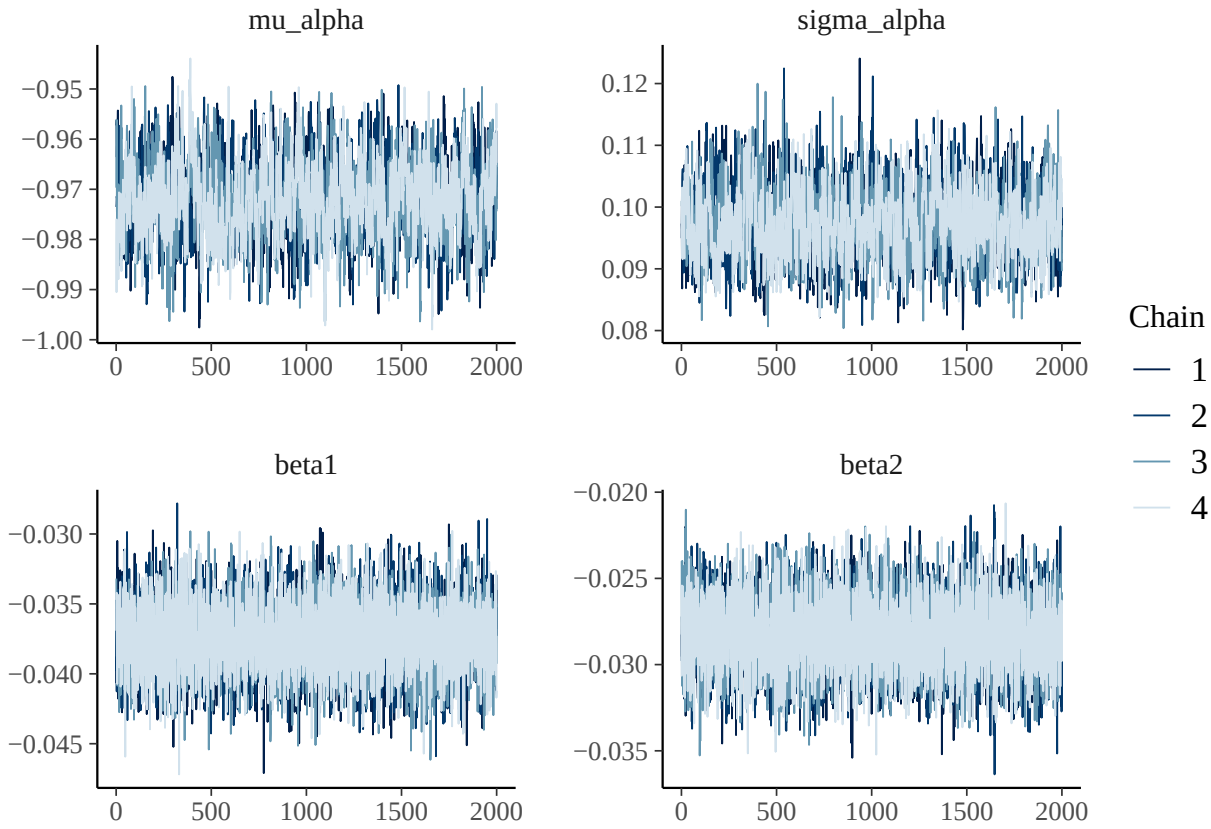
```
    mean(draws_hier_hmc$beta2_lt_0),
    mean(draws_hier_vi_meanfield$beta2_lt_0),
    mean(draws_hier_vi_fullrank$beta2_lt_0),
    mean(draws_hier_laplace$beta2_lt_0)
  )
)

compare_hier_table
```

```
##         method  beta1_mean   beta1_low  beta1_high  beta2_mean   beta2_low
## 1          HMC -0.03737955 -0.04239813 -0.03242582 -0.02822037 -0.03230193
## 2 VI_meanfield -0.03016204 -0.03419361 -0.02597129 -0.03835744 -0.04159581
## 3  VI_fullrank -0.03521280 -0.23733662  0.17359868 -0.04746887 -0.23809374
## 4      Laplace -0.03725717 -0.04166896 -0.03283496 -0.02808238 -0.03111468
##    beta2_high sigma_alpha_mean sigma_alpha_low sigma_alpha_high peak_mean
## 1 -0.02409966       0.09753058      0.08721206       0.10887180  27.15768
## 2 -0.03521808       0.08814715      0.08352667       0.09265795  28.35596
## 3  0.13291138       0.07421289      0.06733014       0.08111863 -46.82520
## 4 -0.02507625       0.09014038      0.08163246       0.09939263  27.16177
##    peak_low peak_high prob_beta1_gt_0 prob_beta2_lt_0
## 1 26.506310  27.71028           0.000           1.000
## 2 28.080787  28.62565           0.000           1.000
## 3 -8.987794  58.01188           0.351           0.694
## 4 26.675853  27.60048           0.000           1.000
```

```
fit_hier_hmc$diagnostic_summary()
```

```
## $num_divergent
## [1] 0 0 0 0
##
## $num_max_treedepth
## [1] 0 0 0 0
##
## $ebfmi
## [1] 0.6107981 0.6435158 0.6526833 0.6781452
```

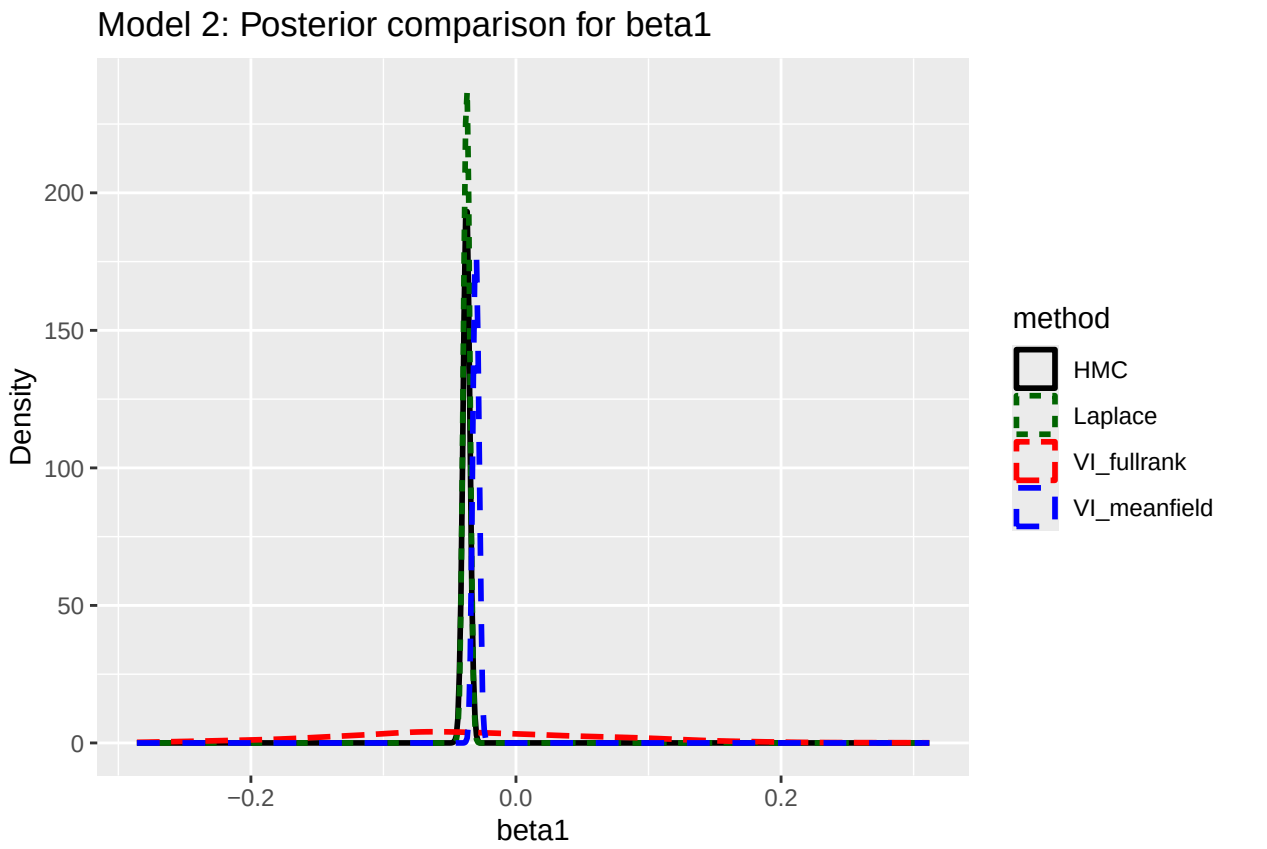
```
mcmc_trace(
  as_draws_array(fit_hier_hmc$draws(c("mu_alpha", "sigma_alpha", "beta1", "beta2")))
)
```



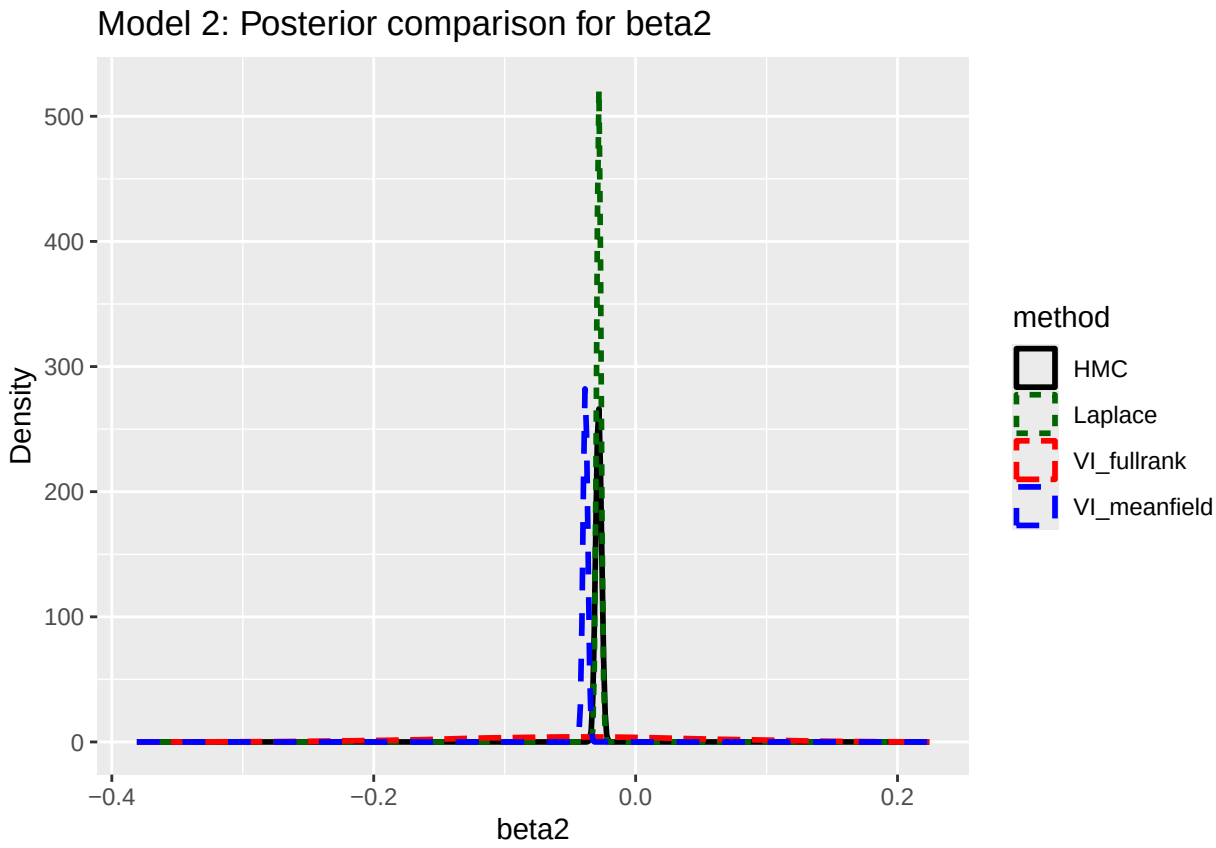

```
method_colors <- c(
  "HMC" = "black",
  "VI_fullrank" = "red",
  "VI_meanfield" = "blue",
  "Laplace" = "darkgreen"
)

plot_df_beta1_hier <- data.frame(
  value = c(
    draws_hier_hmc$beta1,
    draws_hier_vi_meanfield$beta1,
    draws_hier_vi_fullrank$beta1,
    draws_hier_laplace$beta1
  ),
  method = c(
    rep("HMC", nrow(draws_hier_hmc)),
    rep("VI_meanfield", nrow(draws_hier_vi_meanfield)),
    rep("VI_fullrank", nrow(draws_hier_vi_fullrank)),
    rep("Laplace", nrow(draws_hier_laplace))
  )
)

p_beta1 <- ggplot(plot_df_beta1_hier, aes(x = value, color = method, linetype = method)) +
  geom_density(linewidth = 1) +
  scale_color_manual(values = method_colors) +
  labs(
    title = "Model 2: Posterior comparison for beta1",
    x = "beta1",
```

```
    y = "Density"
  )

plot_df_beta2_hier <- data.frame(
  value = c(
    draws_hier_hmc$beta2,
    draws_hier_vi_meanfield$beta2,
    draws_hier_vi_fullrank$beta2,
    draws_hier_laplace$beta2
  ),
  method = c(
    rep("HMC", nrow(draws_hier_hmc)),
    rep("VI_meanfield", nrow(draws_hier_vi_meanfield)),
    rep("VI_fullrank", nrow(draws_hier_vi_fullrank)),
    rep("Laplace", nrow(draws_hier_laplace))
  )
)

p_beta2 <- ggplot(plot_df_beta2_hier, aes(x = value, color = method, linetype = method)) +
  geom_density(linewidth = 1) +
  scale_color_manual(values = method_colors) +
  labs(
    title = "Model 2: Posterior comparison for beta2",
    x = "beta2",
    y = "Density"
  )

plot_df_peak_hier <- data.frame(
  value = c(
    draws_hier_hmc$peak_age,
    draws_hier_vi_meanfield$peak_age,
    draws_hier_vi_fullrank$peak_age,
    draws_hier_laplace$peak_age
  ),
  method = c(
    rep("HMC", nrow(draws_hier_hmc)),
    rep("VI_meanfield", nrow(draws_hier_vi_meanfield)),
    rep("VI_fullrank", nrow(draws_hier_vi_fullrank)),
    rep("Laplace", nrow(draws_hier_laplace))
  )
)

plot_df_peak_hier <- subset(plot_df_peak_hier, value >= 15 & value <= 45)

p_peak <- ggplot(plot_df_peak_hier, aes(x = value, color = method, linetype = method)) +
  geom_density(linewidth = 1) +
  scale_color_manual(values = method_colors) +
  labs(
    title = "Model 2: Posterior comparison for peak age",
    x = "Peak age",
    y = "Density"
  )
```

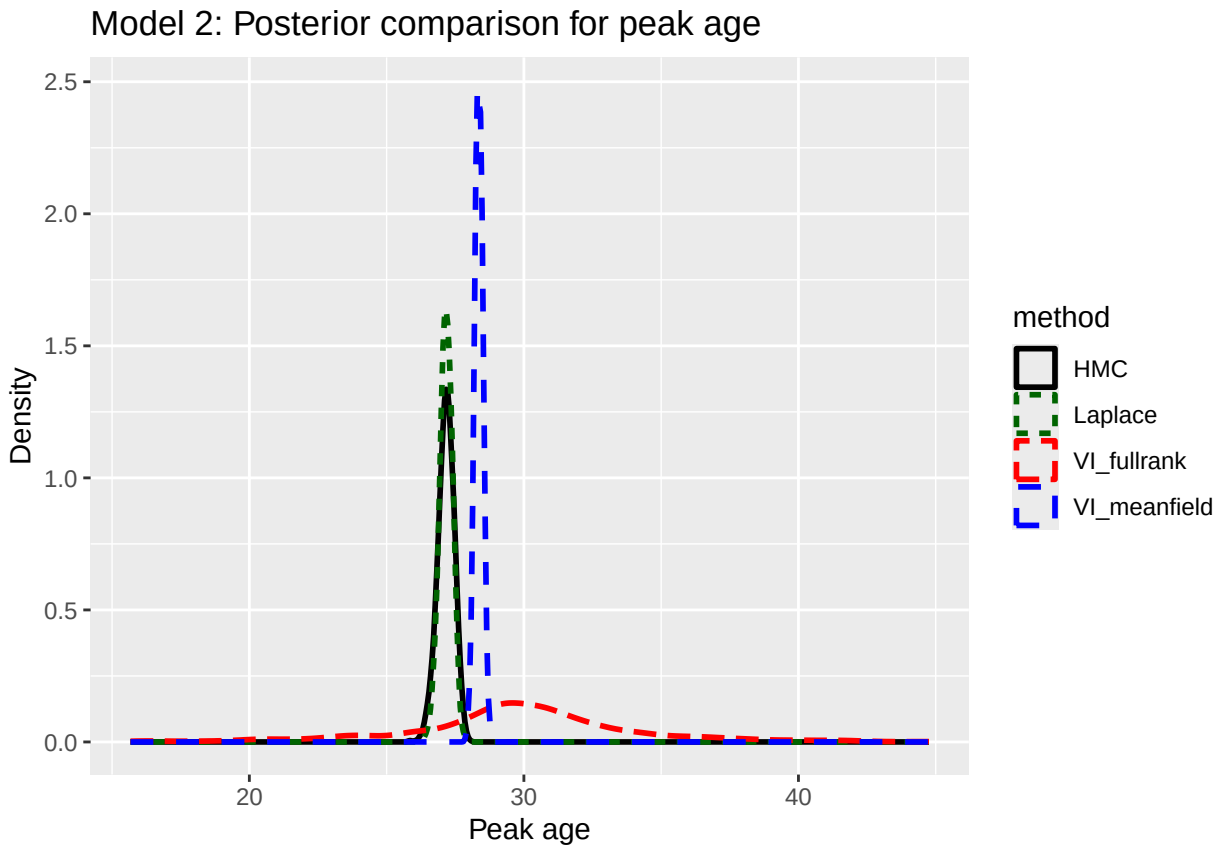
```
p_beta1
```




```
p_beta2
```


Model 2: Posterior comparison for beta2
Density
beta2
500
400
300
200
100
0
−0.4
−0.2
0.0
0.2
method
HMC
Laplace
VI_fullrank
VI_meanfield

```
p_peak
```


Model 2: Posterior comparison for peak age
Density
2.5
2.0
1.5
1.0
0.5
0.0
20
30
40
Peak age
method
HMC
Laplace
VI_fullrank
VI_meanfield